Autonomous Recruitment Task

Assigning desired velocity to an autonomous race car

The solution should be sent as a zip file (name_surname.zip) containing the complete ROS 2 package, including your solution for the velocity_logic.

Additionally, the zip should also contain .png files of both graphs for the best driven lap and a Word document that explains your thought process and ideas.

As a part of the email, write the best achieved lap time.

Department: Autonomous

Car: FSB-RT06D

1. Introduction

The task focuses on a ROS 2 package that roughly simulates the behaviour of an autonomous race car. For this task, the race car is driving on a racetrack that resembles Autodromo Nazionale Monza. The track is shown in the picture below:

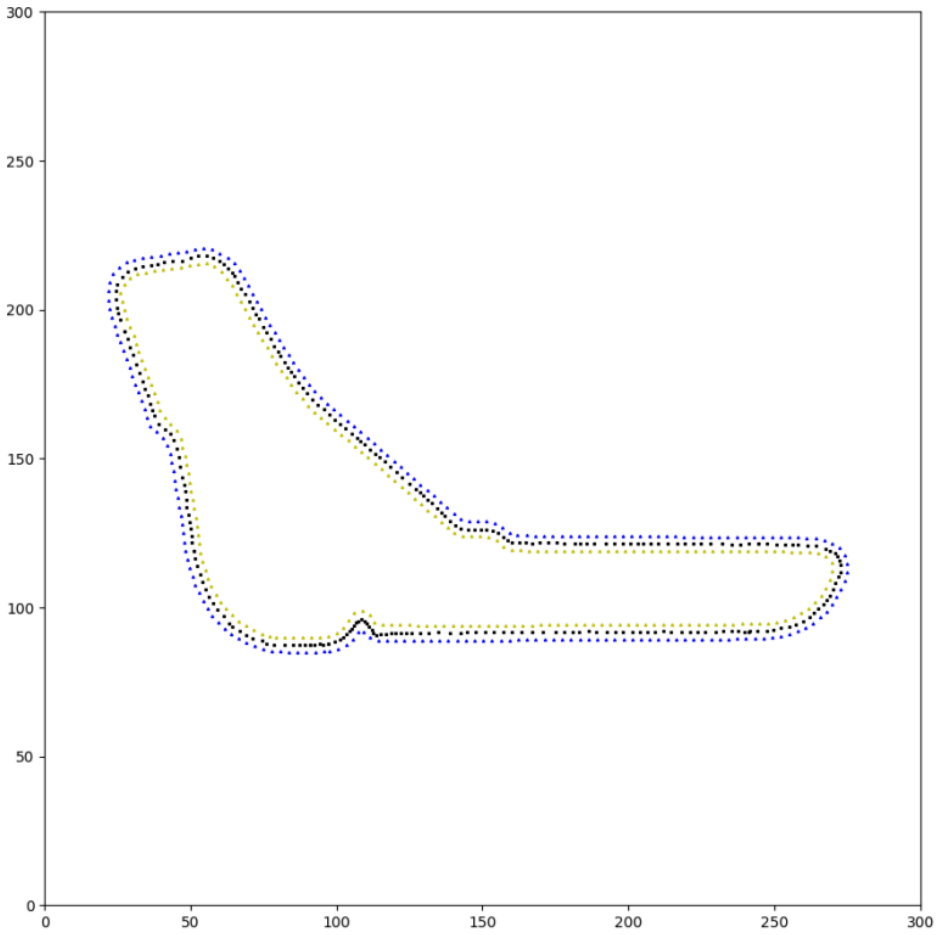


Figure 1 – Racetrack layout

In true Formula Student fashion, the left (outer) track edge is marked with blue cones, while yellow cones are used for the right edge. Black dots represent the checkpoints that the car will attempt to follow, and through which it must drive.

As real autonomous cars have limited vision, you will have access only to the information in the close vicinity.

1.1. Requirements

For this task, it is highly recommended to use:

- Linux Ubuntu version 24.04
- ROS 2 Jazzy Jalisco

2. Task

2.1. Integration

The whole interaction is done through three ROS 2 nodes: controller, simulator and velocity_logic.

The simulator has simple vehicle dynamics implemented, and is responsible for updating the position and orientation of the car based on its angular velocity (yaw rate) and speed.

The controller is responsible for assigning speed and steering commands to the car. It also keeps track of checkpoints that need to be crossed.

Finally, velocity_logic is a helper node that calculates the desired velocity for a given frame. Based on this velocity, the controller will tell the simulator to speed the car up or to slow it down.

All nodes communicate at a frequency of 150 Hz.

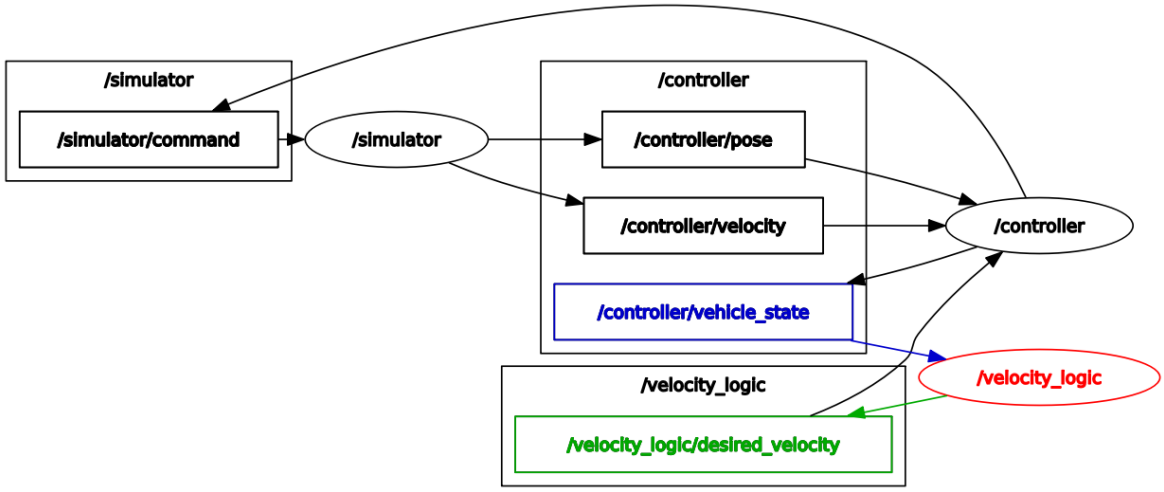


Figure 2 – ROS 2 nodes and topics

Pictured is the communication between the ROS 2 nodes. The nodes are shown in oval-shaped figures, while the topics are shown in rectangles.

After getting the current position and the velocity from the simulator, the controller calculates steering based on the current car orientation from a checkpoint. To determine the velocity command, the controller uses help from the velocity_logic node, which returns the targeted velocity. With both values ready, the controller sends the command to the simulator.

Lastly, the simulator updates the position of the car after applying simple vehicle dynamics. This process is performed 150 times every second.

2.2. Assignment

Your job is to decide what would be the ”ideal” speed for the car in every frame of the explained interaction. In other words, your task is to add logic to the velocity_logic node.

There are two velocity_logic files: velocity_logic.cpp and velocity_logic.py. Except for the programming language used, their functionality is identical. You only need to implement your solution in one of them. **C++ is preferred** and I suggest that you use that if possible, while the Python version is only given as a backup. There are .launch files provided for both versions.

Once you choose the file in which you want to work, you should be able to quickly find the exact place for your code, which is highlighted with comments. Currently, the car is commanded to move at a constant speed of 1.8 m/s.

Command the car by calculating its target speed to get the best possible lap time. Do not change the values in other scripts.

You are provided with this information/inputs:

- scanned_trajectories: a list that contains positions of the next 15 checkpoints in front of the car’s current position.
- current_velocity: current linear velocity of the car.
- current_car_angle: current car orientation in radians with respect to the x-axis
- current_car_position: x and y coordinates of the car's position

Restrictions on the output of a function:

- The car cannot go backward. Assigning a negative speed is prohibited.
- The car must not come to a stop. Assigning a speed of zero is prohibited.
- The car must not be given a command to drive faster than its maximum speed. Assigning a speed above the maximum speed of the car (19.5 m/s) is prohibited
- The car must not have a constant speed.

It is not allowed to use AI (LLM) tools for this task. It is also not allowed to seek help from current or past FSB Racing Team members, as well as other candidates.

Other resources, such as the internet, books, etc., are allowed.

2.3. Usage of graphs

After the simulation is terminated (Ctrl-C), two graphs will be saved:

- Graph that shows speed with respect to time.
- Graph that shows angular speed (yaw rate) with respect to time.

Graphs are saved in a custom format so that they can be redrawn for interactive examination. All graphs are saved in the /drawings directory. You will have to use the provided script to redraw saved graphs. One example of how to start the script from the terminal after positioning in the script's directory can be seen in the picture below. The script takes the full path of the file as an argument.

```
$ python3 redraw_figure.py "/home/matejlovrekovic/
ros2_ws/src/fsbrt_ros_2026/drawings/angular_velocity/
Angular_velocity_2026_10_03 17:14:30.898415.matfig"
```

Figure 3 – Graph command example

2.4. Word document for your ideas

The last part you should provide in your solution is a Word document in which you explain your thought process. Write about your ideas, from the initial ones to the final ones, both successful and unsuccessful. Feel free to implement sketches, graphs, or any other kind of visualization.

2.5. Nice to keep in mind

- The car is modeled so that accelerating from zero to top speed takes about 2.8 seconds and braking from top speed to zero takes about 1.6 seconds.
- Checkpoints are not evenly distributed, so having 15 points in the future does not mean you always have the same length of racetrack part defined. They are also not perfectly centered within cones (consider this a sensor mistake caused by noise).
- Depending on your algorithm, the car may leave the track because of a missed checkpoint. If the car manages to return to the right path, the run is still valid.
- Have greater caution at high speeds, the steering is modeled to be slower linearly as the car speed increases.
- Checkpoint is reached when the car is within a 0.7m radius of that checkpoint.
- After starting the simulation, useful information is printed in the terminal.
- Do not expect each run to be the same because communication cannot follow sampling time exactly.

2.6. Notes

- You can look at the task as if it has two parts: finishing the lap, and finishing the lap as quickly as possible. The second task is where you can really showcase your creativity and dedication.
- The ROS 2 launch file is provided to start the simulation.
- If you get an error about making nodes executable, give all files in the /scripts directory permission to be executable. (tutorial)
- Do not stress about the task, you can always ask for help. At the deadline, send the solution no matter how far you have come. Play around with your ideas and good luck! 😊